

8.2 Continual Learning and Forgetting

Train on task B and task A slips. Replay, freeze, or isolate the change.

These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

Train on task B after task A and **task A's accuracy falls**. That drop is **catastrophic forgetting**. It is not a bug in one optimizer; it is what a single shared W does when the new gradient points away from the old solution.

1. What catastrophic forgetting is

Catastrophic forgetting is sequential training's usual geometry: you train on task B and task A slips. The SFT (or pretrain) minimum for A is a point in weight space. Fine-tuning on B walks away from it. Continual learning is the attempt to add B without burying A.

In an MLP, the same hidden units implemented both functions; XOR then AND in Lab 8 is the cartoon. In an LLM, new instruction data overwrites features that still mattered for coding, or for a language you are not currently sampling. Instruction SFT is a special case: the "old task" is everything the base model already did (code, languages, refusal style).

A single shared W cannot be at two distant minima at once unless the tasks agree. XOR and AND disagree on the same four bit-pairs; that is why Lab 8 is small enough to plot. In an LLM the disagreement is softer (new hospital notes vs old coding skill) but the picture is the same walk in weight space.

This is not "overfitting." Overfitting is train vs test on **one** task. Forgetting is A vs B after a sequential train.

2. Why we use it

You will fine-tune. Production data arrives in waves: a new policy PDF every month, a new hospital corpus, a new tool schema. If you only plot B, you hid the bug. Measuring **A after** B is part of the method.

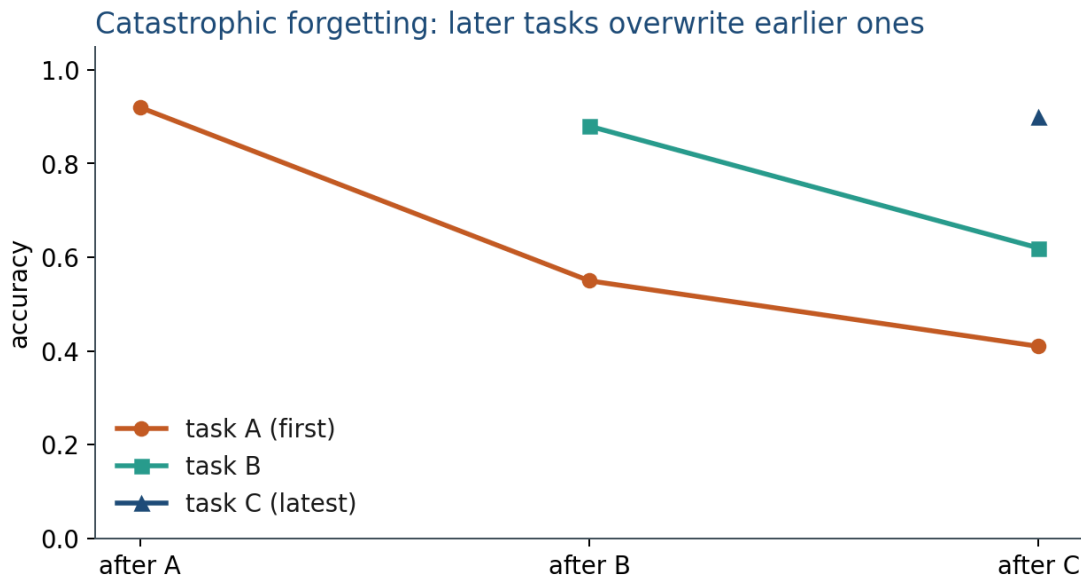
XOR-then-AND in Lab 8 is the cartoon you can plot on a four-point set. The course needs the same habit on instruction collections: keep a **held-out A set** and plot it every k steps of B. If the line is missing from a report, the method is incomplete.

The reason to name forgetting *before* LoRA (note **8.3**) is so the adapter is not a slogan. Replay, freeze, LoRA, and an EWC-style penalty are different claims about whether W moves and whether you still need A's data.

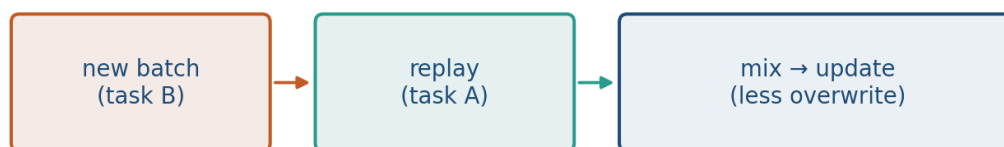
3. Architecture

Same net. There is no extra layer whose job is “memory of A.” Mitigations sit around a shared W :

- **Replay**: mix old batches into the new stream so the gradient still has a component that preserves A.
- **Freeze** early layers, or freeze all of W and train a small head.
- **LoRA** (next note): isolate Δ in BA and leave W frozen.
- **Regularize toward old θ** (EWC cartoon): a penalty on weights that were important for A.



Replay (or freeze, or LoRA) is how you keep task A alive



A full fine-tune is a second copy of W , and that copy walks off task A. Adapters are often the cleaner LLM answer: you add B without moving A's weights. Replay still **moves** W . It only keeps a component of ∇L_A in the batch. LoRA **does not move** W . Those are different claims; do not mix them in one sentence of a report.

A table worth leaving on the board: method | does W move? | need A data? Replay: yes, yes. Frozen LoRA: no, no (for A). EWC: yes, no (needs a Fisher sketch).

4. How it works, step by step

Measure first. Keep a held-out A set. Plot A every k steps of B. If you only report B, you did not look.

Replay. Mix a buffer of A examples into the B batches. The buffer can be real stored rows or **pseudo-replay** (sample the old model and treat those strings as A labels—they are imperfect). A 50/50 mix of A and B in each batch is the simplest version. If batches of 8 mix 2 A examples with 6 B, only a quarter of the data is still A, and a tiny shared hidden layer can still lose XOR.

Freeze or adapt. Smaller learning rates and frozen early layers slow the walk. Attaching a **task adapter** and leaving W frozen (note 8.3) is the isolator: XOR accuracy on the frozen net **cannot** move. That is the thing replay cannot promise.

Regularize. Cartoon EWC adds a quadratic penalty toward θ^A on coordinates that mattered for A. You do not compute a full Fisher in this class; you need the idea that some weights are more expensive to move.

Continual setups you will actually run. Instruction collections arrive in waves (new tool, new hospital, new semester). A legal design is: freeze the base, train LoRA on B, keep the A LoRA. Merging adapters is a separate, lossy step. If you must update one set of weights, replay A on purpose.

5. Mathematical formulas

Cartoon EWC: train B while paying to stay near the A solution on important coordinates,

$$\mathcal{L}_B(\theta) + \frac{\lambda}{2} \sum_i F_i (\theta_i - \theta_i^A)^2.$$

You do not compute a full Fisher in this class; F_i is “how much A cared about weight i .” λ is the knob.

Replay has no extra term: it is just a mixture of losses,

$$\mathcal{L} = \alpha \mathcal{L}_A + (1 - \alpha) \mathcal{L}_B,$$

with α set by how many A rows you put in the batch. LoRA’s claim is different and has no retain loss on W : W is frozen, so A’s accuracy on that frozen net cannot move.

The forgetting gap is a pair of rates, not a slogan. If A had 200 held-out items, 180 correct (90%), and after B only 120 are correct, the gap is 30 points. You still have to check B, or you only measured the damage.

6. Positive points and negative points

Positive.

- Replay is simple and keeps a component of ∇L_A in every batch.
- Pseudo-replay lets you sample the old model when you cannot store A.
- Frozen LoRA (next note) can add B without moving A’s weights at all.

- A held-out A curve turns forgetting from a vibe into a plot.

Negative.

- Replay needs stored data, or imperfect samples from π_A .
- A small A fraction in the batch (2 of 8) can still let XOR die.
- Freeze can block B if the features B needs were in the frozen layers.
- LoRA still forgets if you merge adapters carelessly or train one shared W after all.
- EWC adds knobs (λ , a Fisher sketch) and still moves W .

When not to. Do not report only task-B accuracy after a sequential train. Do not call any accuracy drop “overfitting.” If the tasks must coexist and you can freeze the base, prefer an adapter over hoping replay is enough.

7. Teaching this note

About **30 minutes** at the board: XOR-then-AND on a shared hidden layer, a sketch of accuracy-A vs steps-of-B, then “freeze W , train an adapter.” Play LoRA with the pause on **adapter vs overwrite** (frozen W). Lab 8 section 2 is the tiny forget curve.

Leave a table on the board: method | does W move? | need A data? Replay: yes, yes. Frozen LoRA: no, no (for A). EWC: yes, no (needs a Fisher sketch). Students should be able to fill that without notes.

8. Worked example

Two tasks, one hidden layer of width 2. Train XOR to 100% on a four-point set. Then train AND on the same four (x_1, x_2) bits, same W . AND is linearly separable; XOR is not. After enough AND steps, XOR accuracy typically falls toward chance (2/4). That is forgetting as geometry: the hidden features rotated to serve AND.

Replay: each batch is 2 XOR rows + 2 AND rows. The XOR loss stays in the gradient. You may not hit 100% on both; you will not usually zero XOR.

Adapter: freeze W_{XOR} , learn a LoRA (or a second head) for AND. XOR accuracy on the frozen net **cannot** move. That is the thing replay cannot promise: replay still updates the shared weights.

Plot: x-axis = AND steps {0, 50, 100, 200}, y-axis = XOR held-out accuracy. If the line is missing from a report, the method is incomplete.

Numbers: XOR chance is $2 / 4 = 0.5$. If XOR goes $4 / 4 \rightarrow 2 / 4$ while AND goes $2 / 4 \rightarrow 4 / 4$, you traded tasks. Replay might land at $3 / 4$ and $3 / 4$. LoRA-for-AND keeps XOR at $4 / 4$ on the frozen net.

9. Where students get stuck

- Reporting only task-B accuracy after a sequential train.
 - Calling any accuracy drop “overfitting.” Overfitting is train vs test on **one** task; forgetting is A vs B.
 - Mixing replay with LoRA in the writeup. Say which weights moved.
-

10. Video

[Umar Jamil — LoRA explained](#). Play the opening until W is frozen and only A, B train (first ~10–15 min). Pause on **adapter vs overwrite**: that is this note’s fix. Full LoRA algebra is the next note.

11. Practice

1. You fine-tune only on AND after XOR. What should happen to XOR accuracy if the hidden layer is tiny and shared?
2. Why is sampling the *old* model a form of replay even when you cannot store the old dataset?
3. Name one reason LoRA (next note) can avoid forgetting that replay cannot: the base W never moves.
4. Task A had 200 held-out items, 180 correct (90%). After B, 120 correct. What is the forgetting gap in points, and what must you still check on B?
5. Batches of 8. Replay mixes 2 A examples with 6 B. What fraction of the gradient’s data is still A, and why might XOR still die if that fraction is this small?